

Delcom CI/CD & Submission CI/CD

💖 Note

Panduan ini merupakan langkah untuk mengimplementasikan CI/CD dari aplikasi yang telah dikembangkan dengan memanfaatkan layanan Delcom.

🌸 A. Integrasi CI/CD

🌻 1.1 Github

1.1.1 Repository Aplikasi di Github

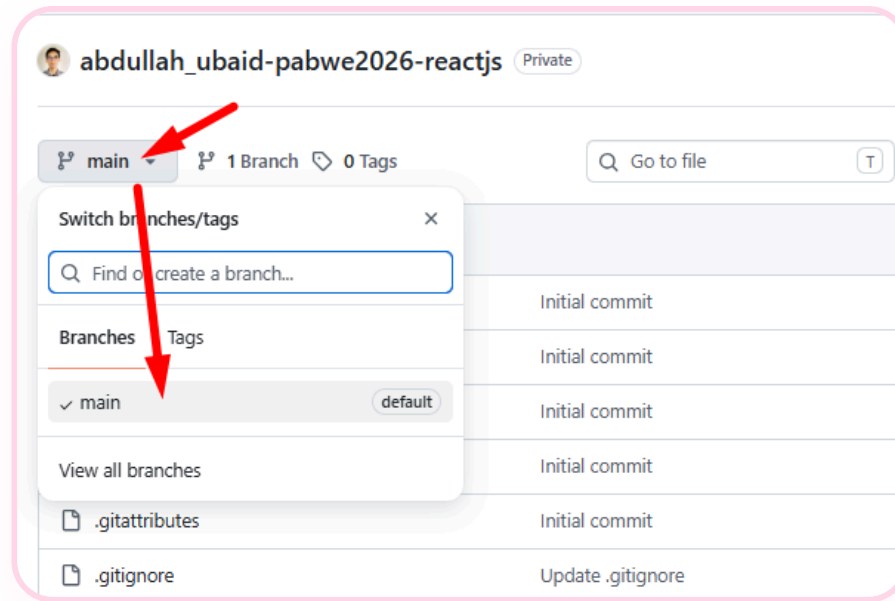
Pastikan sudah ada kode program aplikasi yang ingin di terapkan CI/CD. Jika sudah ada buat repositori Github untuk kode pogram tersebut.

The screenshot shows a GitHub repository interface for 'abdullah_ubaid-pabwe2026-reactjs' (Private). The repository has 1 Branch and 0 Tags. The file list includes:

File/Folder	Commit Message	Time
public	Initial commit	4 minutes ago
src	Initial commit	4 minutes ago
.env	Initial commit	4 minutes ago
.env.example	Initial commit	4 minutes ago
.gitattributes	Initial commit	4 minutes ago
.gitignore	Update .gitignore	2 minutes ago
.oxlintc.json	Initial commit	4 minutes ago
README.md	Update README.md	now
bun.lock	Initial commit	4 minutes ago
index.html	Initial commit	4 minutes ago
package.json	Initial commit	4 minutes ago
vite.config.js	Initial commit	4 minutes ago

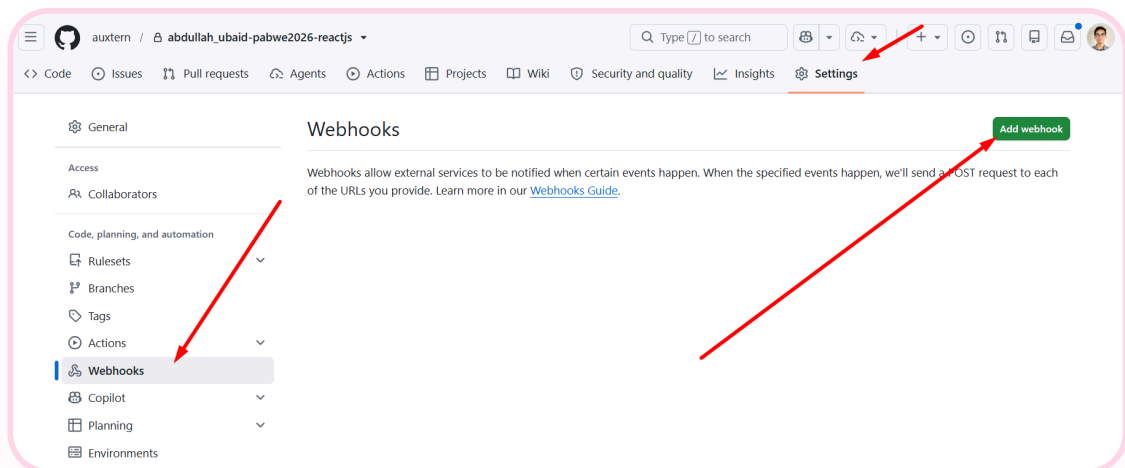
1.1.2 Melihat Branch Default

Pada website Github pilih opsi **Branch** maka akan terlihat branch default dalam kasus ini adalah branch **main**.



1.1.3 Menambahkan Webhook

1. Pada repo github aplikasi kamu pilih tab **Settings**. Pada halaman pengaturan di sebelah kiri cari dan pilih pada menu **webhooks**. Setelah itu pilih tombol **Add webhook**.



2. Pada halaman tambah webhook, di bagian **Payload URL** isi dengan url berikut: **https://jenkins.delcom.org/github-webhook/**. Pada **Content type**

pilih opsi **application/json**.

Webhooks / Add webhook

We'll send a POST request to the URL below with details of any subscribed events. You can also specify which data format you'd like to receive (JSON, x-www-form-urlencoded, etc). More information can be found in [our developer documentation](#).

Payload URL *

Content type *

Secret

3. Scrool ke bawah centan pada opsi **Enable SSL verification** dan **Just the push event**. Centang juga pada opsi **Active**. Setelah itu pilih tombol **Add webhook**.

SSL verification

By default, we verify SSL certificates when delivering payloads.

☒ **Enable SSL verification** ☐ **Disable (not recommended)**

Which events would you like to trigger this webhook?

☒ **Just the push event.**

☐ **Send me everything.**

☐ **Let me select individual events.**

☒ **Active**

We will deliver event details when this hook is triggered.

Add webhook

4. Jika berhasil akan tampil data webhook sesuai yang telah ditambahkan sebelumnya.

Webhooks **Add webhook**

Webhooks allow external services to be notified when certain events happen. When the specified events happen, we'll send a POST request to each of the URLs you provide. Learn more in our [Webhooks Guide](#).

✓ <https://jenkins.delcom.org/github-w...> (push)

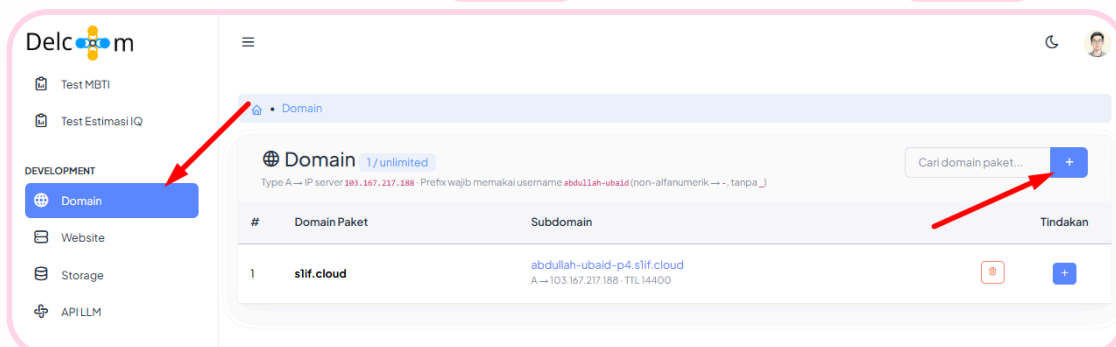
Last delivery was successful.

Edit **Delete**

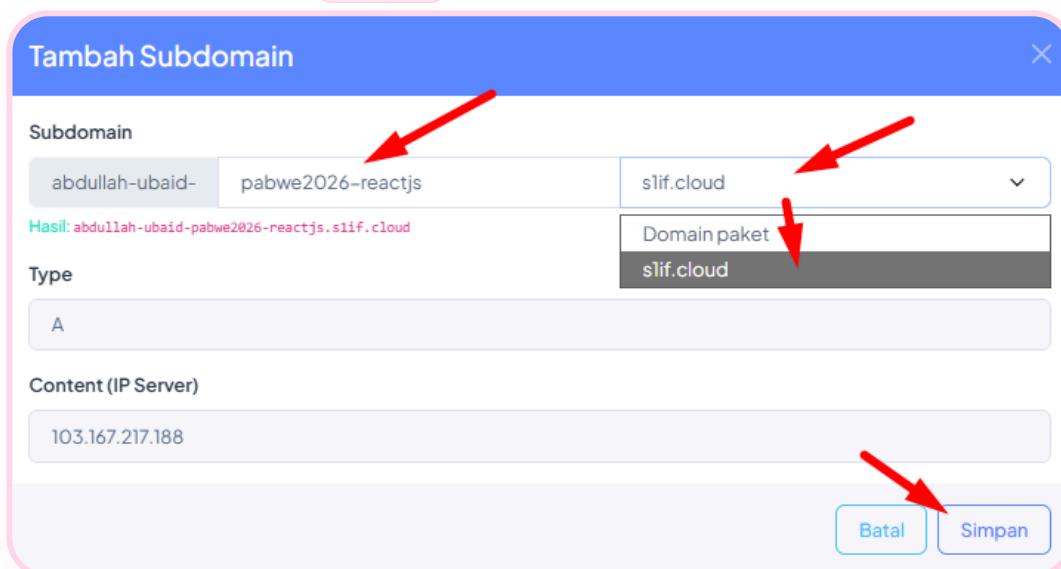
1.2 Delcom

1.2.1 Menambahkan Domain

1. Pada halaman Delcom pilih menu **Domain**. Setelah itu pilih tombol **Tambah**.



2. Pada modal yang tampil pada bagian nama domain isi sesuai nama domain yang kamu butuhkan. Pada bagian pemilihan domain paket pilih pada opsi domain yang tersedia. Setelah itu pilih tombol **Simpan**.

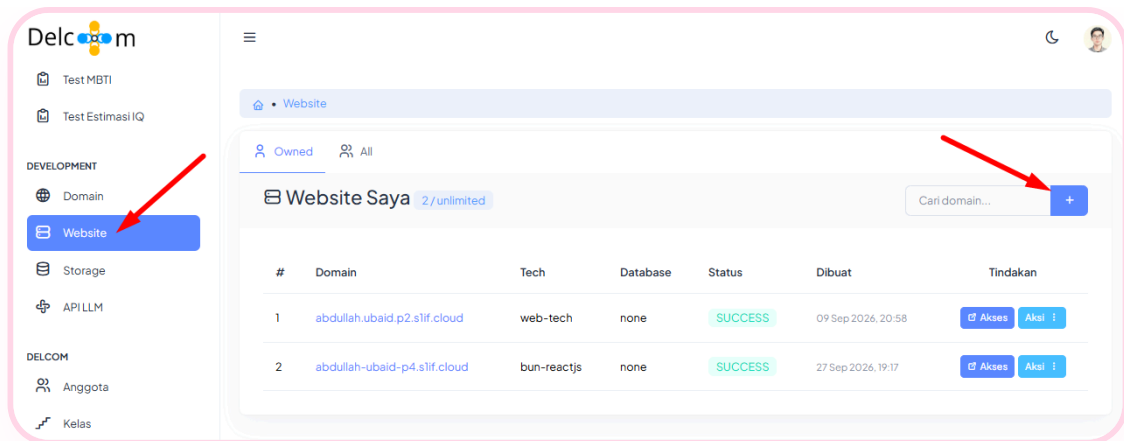


3. Jika berhasil akan tampil domain yang baru saja ditambahkan sebelumnya.



1.2.2 Menambahkan Website

1. Pada halaman Delcom pilih menu **Website**. Setelah itu pilih tombol **Tambah**.



2. Pada modal yang tampil di bagian **pilih domain** pilih domain yang baru ditambahkan atau domain lain sesuai kebutuhan. Pada bagian **teknologi** isi sesuai teknologi yang digunakan pada aplikasi sebagai contoh **bun-reactjs** artinya menggunakan ReactJS dan Bun JS runtime. Pada bagian **database** pilih **none** jika tidak menggunakan database. Pada bagian **sumber** pilih **none** untuk integrasi dengan CI/CD. Setelah itu pilih tombol **Deploy**.

+ Tambah Website

Pilih Domain *

abdullah-ubaid-pabwe2026-reactjs.s1if.cloud

Paket Domain hanya subdomain ber-prefix username-mu yang belum dipakai website.

Domain

abdullah-ubaid-pabwe2026-reactjs.s1if.cloud

Teknologi *

bun-reactjs

Database *

none

Sumber *

None

☐ Izinkan replace jika domain sudah ada

Batal Deploy

3. Jika berhasil akan tampil website yang baru saja ditambahkan sebelumnya.

Owned All

Website Saya 3/unlimited

Cari domain... +

#	Domain	Tech	Database	Status	Dibuat	Tindakan
1	abdullah.ubaid.p2.slif.cloud	web-tech	none	SUCCESS	09 Sep 2026, 20:58	Akses Aksi
2	abdullah-ubaid-p4.slif.cloud	bun-reactjs	none	SUCCESS	27 Sep 2026, 19:17	Akses Aksi
3	abdullah-ubaid-pabwe2026--reactjs.slif.cloud	bun-reactjs	none	IN_PROGRESS	29 Sep 2026, 17:02	Akses Aksi

1.2.3 Melihat Konfigurasi Website

- Pilih website yang ingin dilihat konfigurasinya.

Owned All

Website Saya 3/unlimited

Cari domain... +

#	Domain	Tech	Database	Status	Dibuat	Tindakan
1	abdullah.ubaid.p2.slif.cloud	web-tech	none	SUCCESS	09 Sep 2026, 20:58	Akses Aksi
2	abdullah-ubaid-p4.slif.cloud	bun-reactjs	none	SUCCESS	27 Sep 2026, 19:17	Akses Aksi
3	abdullah-ubaid-pabwe2026--reactjs.slif.cloud	bun-reactjs	none	SUCCESS	29 Sep 2026, 17:02	Akses Aksi

- Akan terlihat konfigurasi deploy dari website yang bisa digunakan untuk integrasi CI/CD.

abdullah-ubaid-pabwe2026--reactjs.slif.cloud SUCCESS bun-reactjs none Akses Aksi

Info .env Nginx Logs Database

Domain: abdullah-ubaid-pabwe2026--reactjs.slif.cloud Website ID: 5cd6b2c0-14d3-41e4-8e21-04e65dca1861

Token Access: fe1e097406d3cc0121d974dc0c9d1c386e0c3c2c26737d15

Konfigurasi Deploy

```

URL_PROGRESS=https://delcom.org/api/dws/websites/progress
URL_REDEPLOY=https://delcom.org/api/dws/websites/redeploy
WEBSITE_ID=5cd6b2c0-14d3-41e4-8e21-04e65dca1861
DEPLOY_TOKEN=fe1e097406d3cc0121d974dc0c9d1c386e0c3c2c26737d15
  
```

Klik sekali untuk select all, lalu salin (Ctrl+C / Cmd+C).

Teknologi: bun-reactjs Database: none

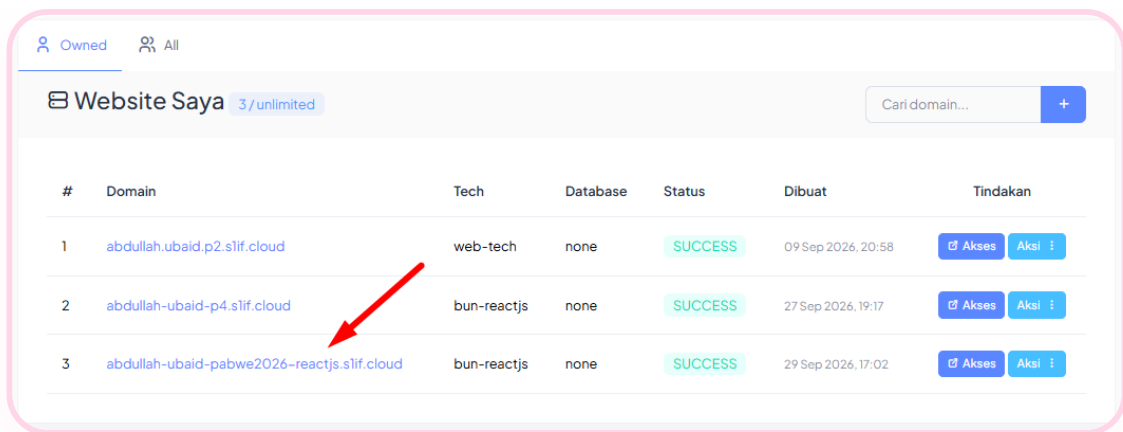
Sumber: none Source URL:

Source Secret: Tidak ada Creator: @abdullah.ubaid, Abdullah Ubaid, S.Kom

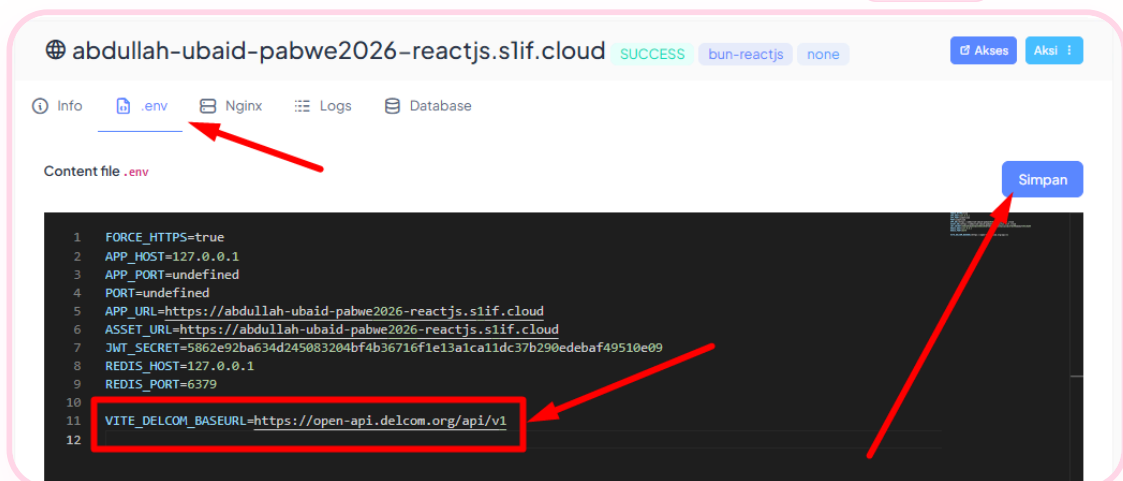
Dibuat: 29 Sep 2026, 17:02

1.2.4 Mengubah File .env Website

- Pilih website yang ingin diubah nilai `.env`-nya.



2. Pada halaman website pilih tab **.env**, selanjutnya pada konten file tambahkan konfigurasi yang dibutuhkan oleh aplikasi. Setelah itu pilih tombol **Simpan**.

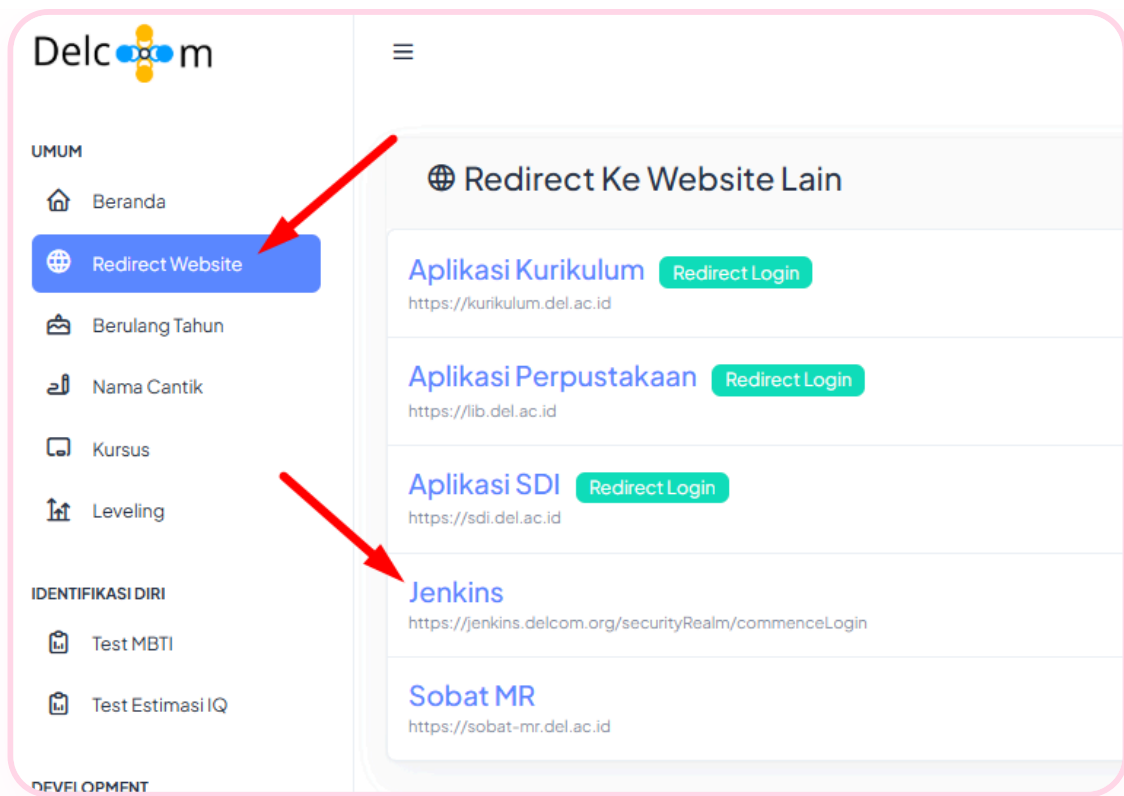


3. Deployment telah siap digunakan.

🌻 1.3 Jenkins

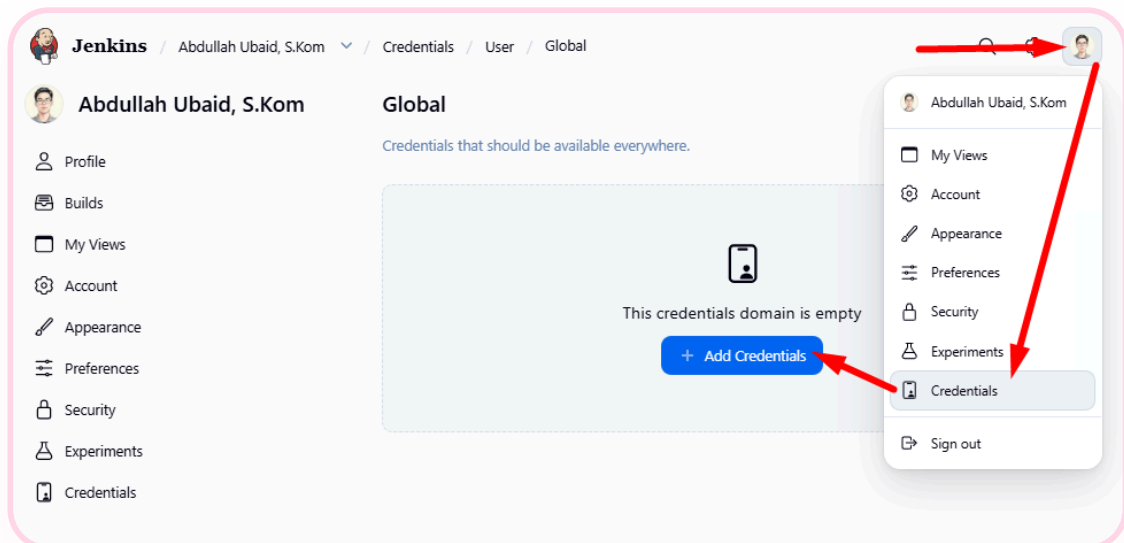
1.3.1 Akses Jenkins

1. Buka menu **Redirect website**. Pilih website **Jenkins**.



1.3.2 Menambahkan credentials

1. Pada halaman jenkins pilih photo profile, pada daftar menu yang tampil pilih **Credentials**. Setelah itu pilih tombol **Add Credentials**.



2. Pada dialog yang tampil pilih tipe kredensialnya yang **Username with password**.

Setelah itu pilih tombol **Next**.

Add Credentials

Select a type of credential

- Username with password**
Commonly used for authentication to services like Git, APIs, or registries.
- GitHub App
- GitHub Token
- SSH Username with private key
- Secret file
- Secret text
- X.509 Client Certificate
- Certificate
Upload a PKCS#12 or PEM encoded certificate and private key.

Next

3. Pada modal yang tampil di bagian username isi dengan username github kamu, centang pada opsi **Treat username as secret**, pada bagian password isi dengan **token github kamu** lihat cara mendapatkannya pada video berikut: <https://youtu.be/ypDK40QAuQo>. Pada bagian ID isi dengan **github-{username-github-kamu}**. Contoh: **github-auxtern**. Setelah itu pilih tombol **Create**.

Add Username with password

Username

auxtern

☒ Treat username as secret ?

Password

.....

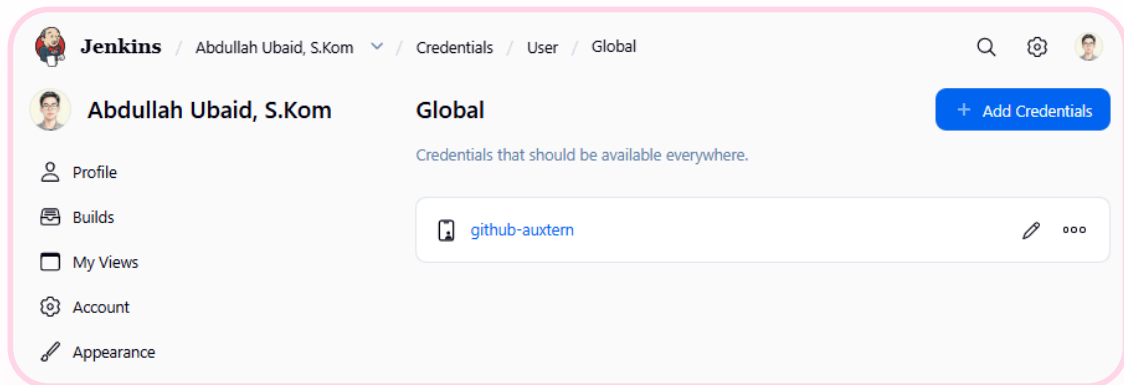
ID ?

github-auxtern

Description ?

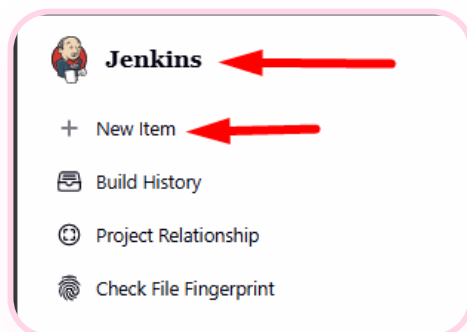
Create

4. Jika berhasil akan terlihat **credentials** yang barus saja ditambahkan.



1.3.3 Menambahkan Item Baru

1. Pilih logo Jenkins di bagian kanan atas. Setelah itu pilih menu **New Item**.



2. Pada isian **enter an item name** samakan saja dengan nama repository Github.
Pada pilihan **select an item type** pilih opsi **Pipeline**. Setelah itu pilih

tombol **Ok**.

New Item

Enter an item name

Select an item type

Pipeline
 Build, test, and deploy using pipelines. Supports stages, parallel work, and running on multiple agents.

Freestyle project
 Classic, general-purpose job type that checks out from up to one SCM, executes build steps serially, followed by post-build steps like archiving artifacts and sending email notifications.

Multi-configuration project
 Suitable for projects that need a large number of different configurations, such as testing on multiple environments, platform-specific builds, etc.

Folder
 Creates a container that stores nested items in it. Useful for grouping things together. Unlike view, which is just a filter, a folder creates a separate namespace, so you can have multiple things of the same name as long as they are in different folders.

Multibranch Pipeline
 Creates a set of Pipeline projects according to detected branches in one SCM repository.

Organization Folder
 Creates a set of multibranch project subfolders by scanning for repositories.

Duplicate an existing item
 Create a new item by copying the configuration of an existing one.

OK

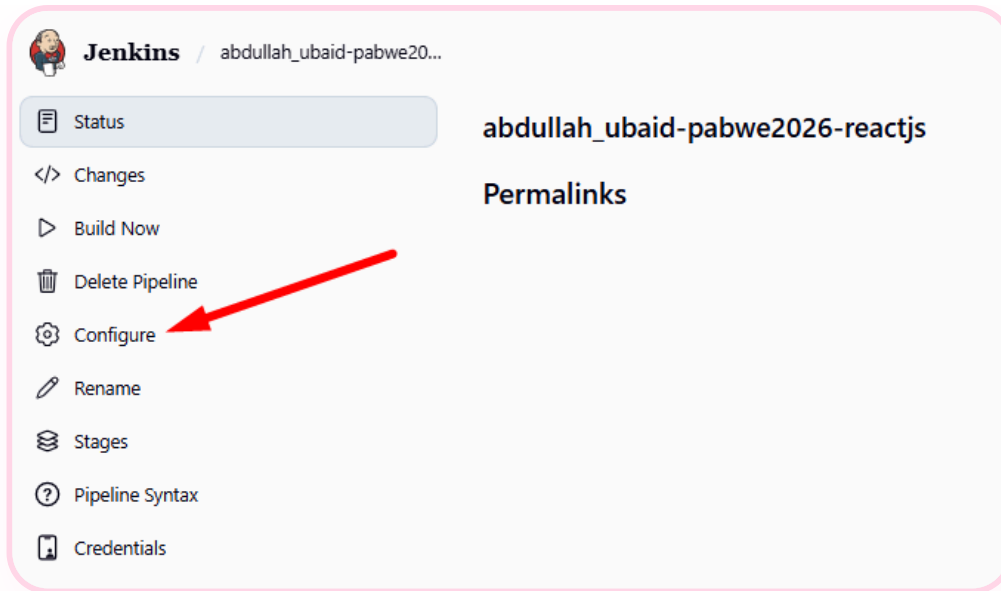
3. Akan terlihat item yang baru saja ditambahkan. Klik pada judul item tersebut.

All +

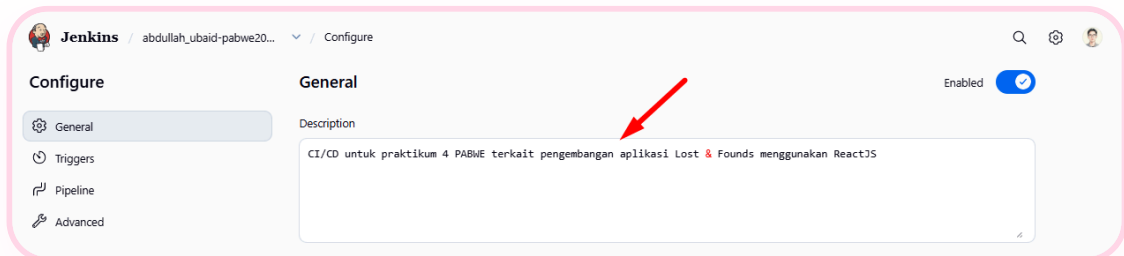
S	W	Name ↓	Last Success	Last Failure	Last Duration	# Issues
✓	☁	abdullah_ubaid-pabwe2026-p4	1 day 21 hr #15	1 day 21 hr #14	57 sec	0
...	☀	abdullah_ubaid-pabwe2026-reactjs	N/A	N/A	N/A	-
...	☀	admin-app	N/A	N/A	N/A	-
✓	☁	itdeveloper_todo-app	6 days 21 hr #11	23 days #8	1 min 19 sec	0

1.3.4 Melakukan Konfigurasi Item

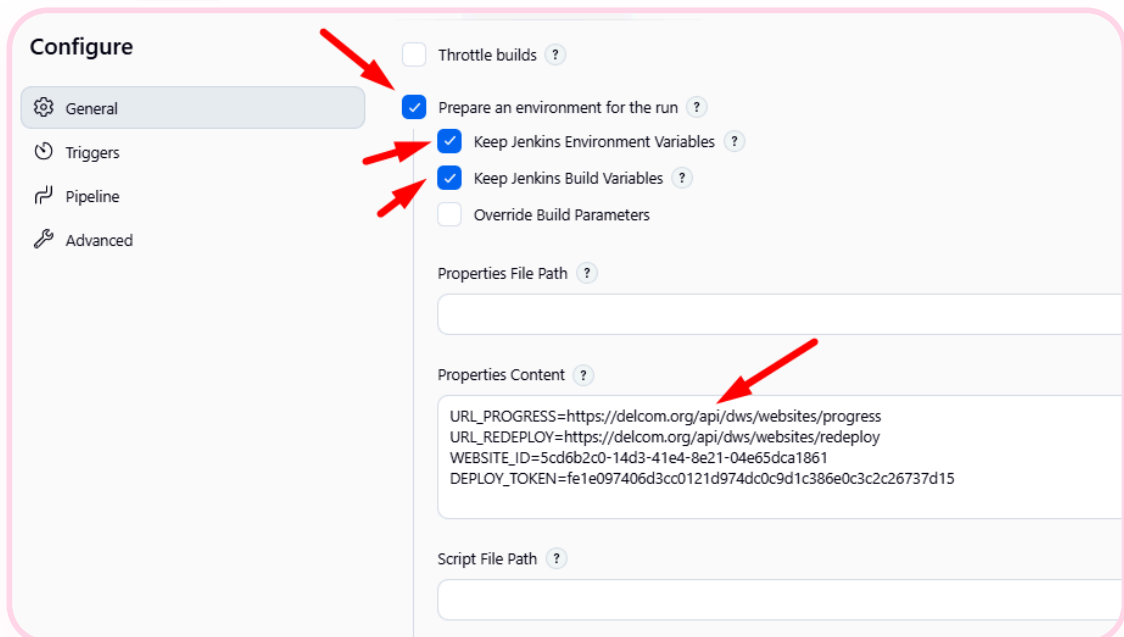
1. Pada halaman item pilih menu **Configure**.



2. Pada halaman konfigurasi di bagian deskripsi isi dengan teks **CI/CD untuk praktikum 4 PABWE terkait pengembangan aplikasi Lost & Founds menggunakan ReactJS**.



3. Scroll ke bawah dan centang pada opsi **Prepare an environment for the run**. Pada bagian **properties content** isi sesuai konfigurasi deploy pada tahapan **1.2.3**.



4. Scroll ke bawah pada bagian **Trigger** centang opsi **GitHub hook trigger**

for GITScm polling .

Configure

General

Triggers

Pipeline

Advanced

Triggers

Set up automated actions that start your build based on specific events, like code changes or scheduled times.

☐ Build after other projects are built ?

☐ Build periodically ?

☒ GitHub hook trigger for GITScm polling ?

☐ Poll SCM ?

☐ Trigger builds remotely (e.g., from scripts) ?

5. Pada bagian **Pipeline** di bagian **Defenition** pilih opsi **Pipeline script from SCM** . Pada opsi **SCM** pilih opsi **Git** . Pada bagian **Repository URL** isi dengan url repositori aplikasi github kamu. Pada bagian **Credentials** pilih opsi sesuai kredensial yang telah ditambahkan pada bagian **1.3.2** . Pada bagian **Branch Specifier** isi sesuai dengan branch default repo kamu lihat pada tahapan **1.1.2** .

Configure

General

Triggers

Pipeline

Advanced

Pipeline

Define your Pipeline using Groovy directly or pull it from source control.

Definition

Pipeline script from SCM

SCM

Git

Repositories

Repository URL

https://github.com/auxtern/abdullah_ubaid-pabwe2026-reactjs

Credentials

Owner - github-auxtern

+ Add

Advanced

+ Add Repository

Branches to build

Branch Specifier (blank for 'any')

*/main

+ Add Branch

6. Scroll ke bawah pada bagian **Script path** pastikan nilainya adalah **Jenkinsfile** . Centang juga opsi **Lightweight checkout** .

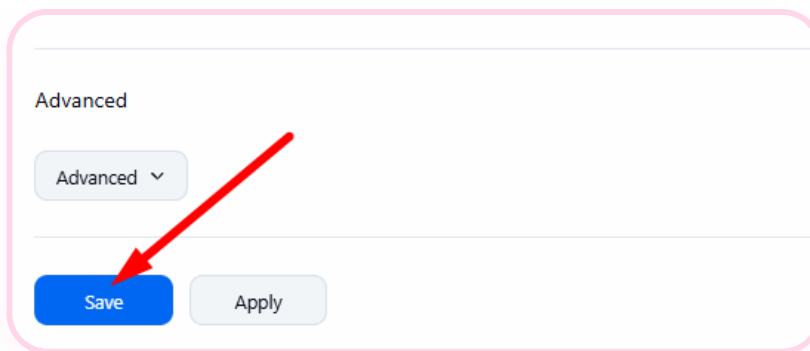
Script Path

Jenkinsfile

☒ Lightweight checkout ?

[Pipeline Syntax](#)

7. Setelah konfigurasi selesai dilakukan pilih tombol **Save** .

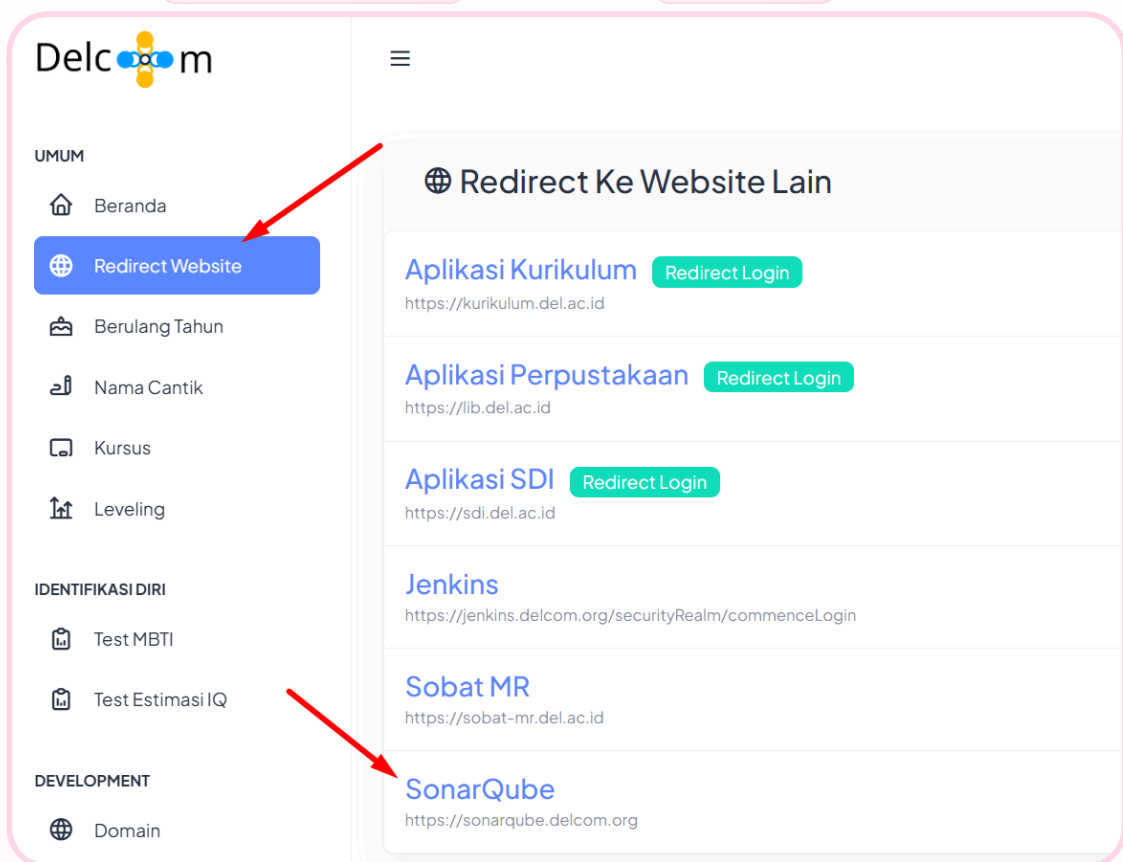


8. Jenkins telah siap digunakan.

1.4 SonarQube

1.4.1 Akses SonarQube

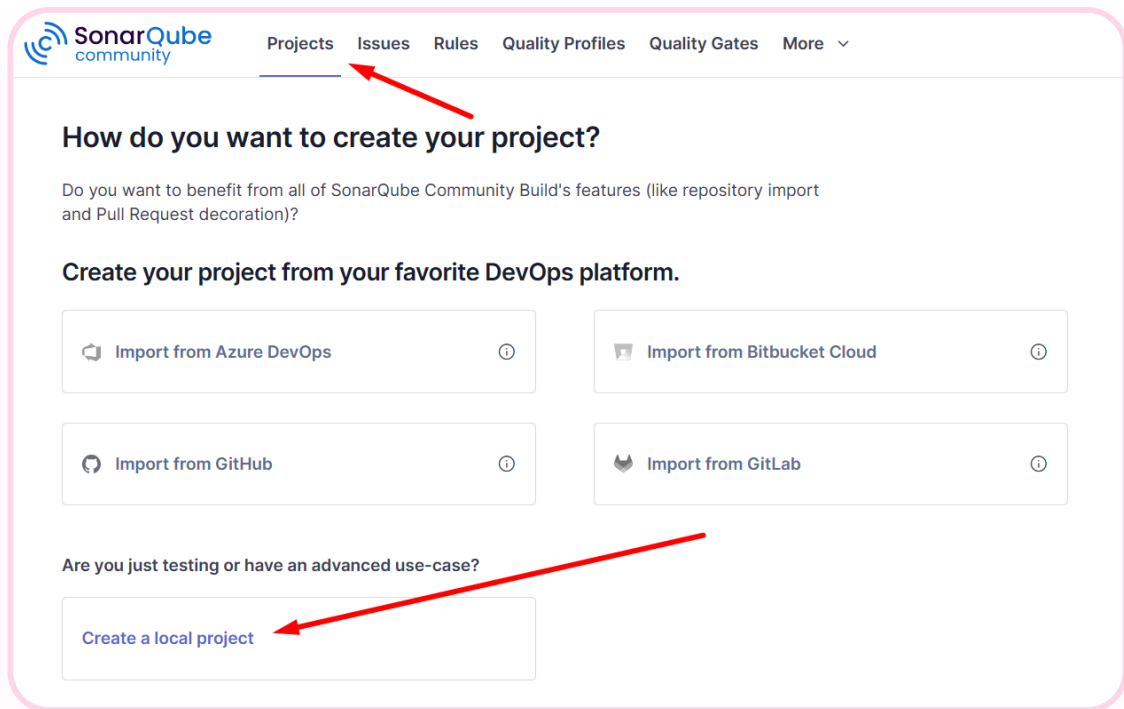
1. Buka menu **Redirect website**. Pilih website **SonarQube**.



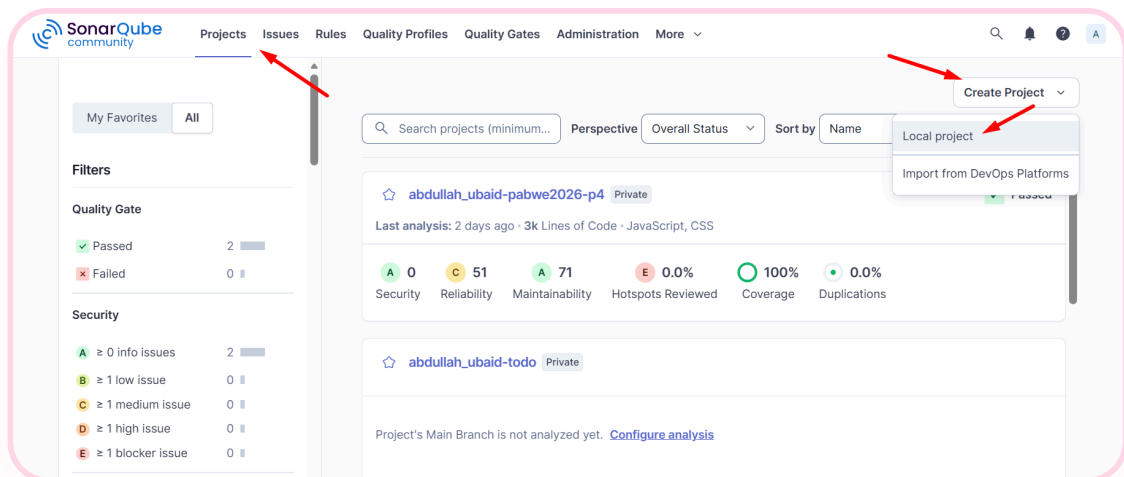
1.4.2 Menambahkan Proyek Baru

1. Pada halaman **SonarQube** pilih tab **Projects**. Setelah itu pilih menu **Create a local project** untuk yang belum ada proyek. Jika sudah ada proyek, pada opsi pilihan **Create project** pilih menu **Local project**.

Tampilan jika belum ada proyek



Tampilan jika sudah ada proyek



2. Pada halaman tambah proyek, di bagian **project display name** dan **project key** isi sesuai nama repo aplikasi github kamu. Di bagian **Main branch name** isi sesuai dengan branch default repo kamu lihat pada tahapan **1.1.2**. Setelah itu pilih

tombol **Next**.

1 of 2

Create a local project

Project display name * ⓘ

abdullah_ubaid-pabwe2026-reactjs

Project key * ⓘ

abdullah_ubaid-pabwe2026-reactjs

Main branch name *

main

The name of your project's default branch [Learn More](#) ⓘ

Cancel **Next**

3. Di tahapan pembuatan proyek brikutnya, pilih opsi **Follows the instances default**. Setelah itu pilih tombol **Create project**.

2 of 2

Set up new code for project

The new code definition sets the criteria for what's considered new code, allowing you to focus on the most recent changes in your project.

This setting allows project administrators to customize the new code definition for their project. [Learn more in documentation](#) ⓘ

Follows the instance's default

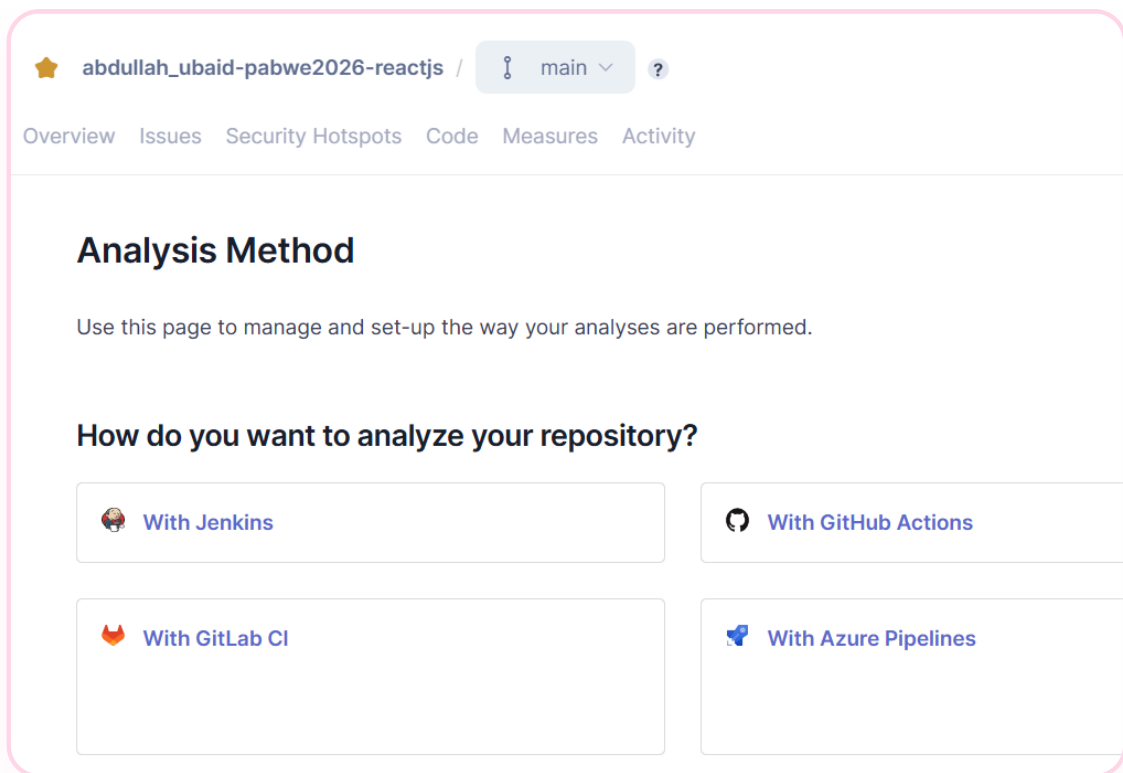
Current default: Previous version

Is custom

Choose whether it should be the **previous version**, a **number of days** or a **reference branch**

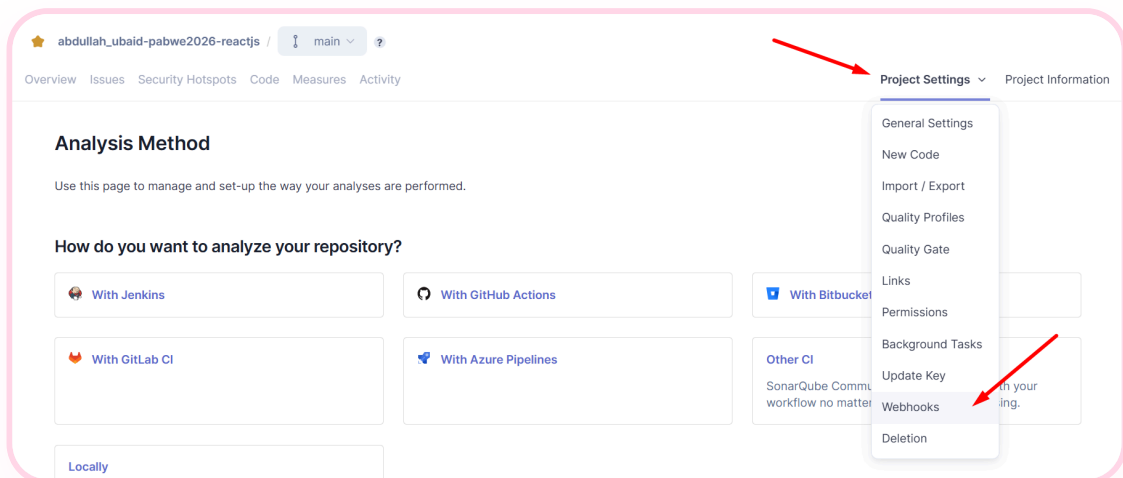
Back **Create project**

4. Tampilan proyek SonarQube setelah berhasil dibuat.

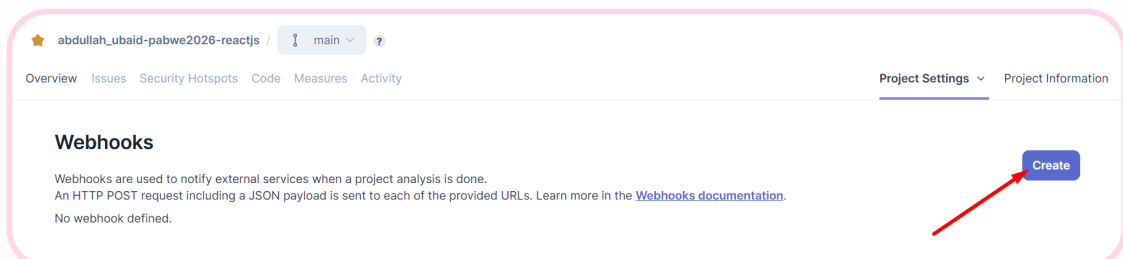


1.4.3 Menambahkan Webhook

1. Pada halaman proyek SonarQube pilih opsi **Project Settings**. Pada daftar menu yang tampil pilih menu **webhooks**.



2. Pada halaman webhooks pilih tombol **Create**.



3. Pada modal tambah webhook, di bagian **Name** isi dengan nilai **Jenkins**, di bagian **URL** isi dengan nilai **http://cicd-jenkins:8080/sonarqube-webhook/**.

Setelah itu pilih tombol **Create**.

Create Webhook

Name *

URL *

Server endpoint that will receive the webhook payload, for example: "http://my_server/foo". If HTTP Basic authentication is used, HTTPS is recommended to avoid man in the middle attacks. Example: "https://myLogin:myPassword@my_server/foo"

Secret

If provided, secret will be used as the key to generate the HMAC hex (lowercase) digest value in the 'X-Sonar-Webhook-HMAC-SHA256' header.

Create **Cancel**

4. Jika berhasil akan tampil data webhook yang baru saja ditambahkan sebelumnya.

★
abdullah_ubaid-pabwe2026-reactjs
/
main
?

Overview Issues Security Hotspots Code Measures Activity

Project Settings
Project Information

Webhooks

Webhooks are used to notify external services when a project analysis is done. An HTTP POST request including a JSON payload is sent to each of the provided URLs. Learn more in the [Webhooks documentation](#).

Create

Name	URL	Has secret?	Last delivery	Actions
Jenkins	http://cicd-jenkins:8080/sonarqube-webhook/	No	Never	⋮

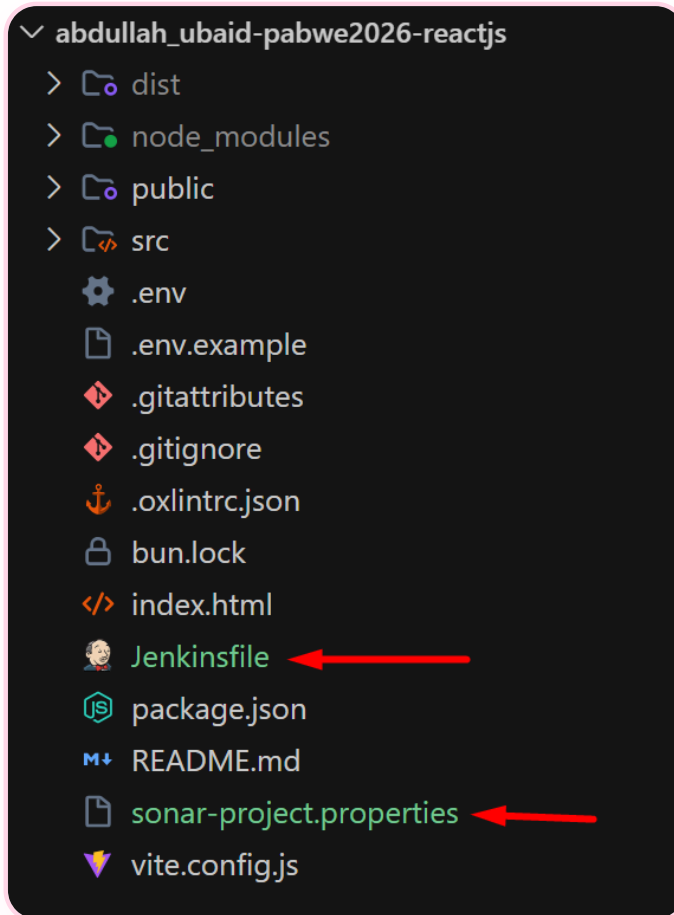
5. SonarQube telah siap digunakan.

🌻 1.5 Melakukan Integrasi CI/CD

1.5.1 Menambahkan konfigurasi

1. Tambahkan file **Jenkinsfile** dan **sonar-project.properties** pada proyek

kamu.



2. Modifikasi isi file **Jenkinsfile**, seperti berikut:

Coding Time

```

1 pipeline {
2
3   agent any
4
5   options {
6     timestamps()
7     skipDefaultCheckout(true)
8     // Stop later stages if a publisher marks the build UNSTABLE/FAILURE mid-run
9     skipStagesAfterUnstable()
10  }
11
12  stages {
13
14    // =====
15    // CHECKOUT
16    // =====
17    stage('checkout') {
18      agent {
19        docker {
20          image 'oven/bun:alpine'
21          reuseNode true
22        }
23      }
24
25      steps {
26        checkout scm

```

```
27     }
28   }
29
30   // =====
31   // INSTALL DEPENDENCIES
32   // =====
33   stage('Install Dependencies') {
34     agent {
35       docker {
36         image 'oven/bun:alpine'
37         reuseNode true
38       }
39     }
40
41     steps {
42       sh '''
43         set -e
44
45         echo "=== Installing Dependencies ==="
46
47         bun install
48
49         echo "=== Dependencies Installed ==="
50       '''
51     }
52   }
53
54   // =====
55   // TEST
56   // =====
57   stage('Test') {
58     agent {
59       docker {
60         image 'node:24-alpine'
61         reuseNode true
62       }
63     }
64
65     steps {
66       sh '''
67         set -e
68
69         echo "=== Running Tests with Coverage ==="
70
71         npx vitest run --coverage
72
73         echo "=== Tests Passed ==="
74       '''
75     }
76   }
77
78   // =====
79   // TRIVY SECURITY SCAN
80   // =====
81   stage('Trivy Security Scan') {
82     agent {
83       docker {
84         image 'aquasec/trivy:0.74.0'
85         reuseNode true
```

```

86
87     args '''
88         --entrypoint=""
89         -e HOME=/tmp
90         -e XDG_CACHE_HOME=/tmp/.cache
91     '''
92 }
93 }
94
95 steps {
96     sh '''
97         set -e
98
99         mkdir -p .trivy-cache || true
100
101         echo "=====
102         echo "          TRIVY SECURITY SCAN"
103         echo "=====
104
105         echo "=== Trivy Version ==="
106         trivy --version
107
108         echo "=== Trivy Scan ==="
109
110         trivy fs \
111             --cache-dir .trivy-cache \
112             --scanners vuln \
113             --severity HIGH,CRITICAL \
114             --format sarif \
115             --output trivy-results.sarif \
116             --exit-code 1 \
117             .
118
119         echo "=== Trivy Result ==="
120         ls -lh trivy-results.sarif
121     '''
122 }
123
124 post {
125     always {
126         // failOnError must be false: otherwise Warnings NG can mark the
127         // whole build FAILURE while later stages still run (all green, badge
red).
128
129         // Build failure on HIGH/CRITICAL comes from trivy --exit-code 1 above.
130         recordIssues(
131             enabledForFailure: true,
132             failOnError: false,
133             tools: [
134                 sarif(
135                     id: 'trivy',
136                     name: 'Trivy Security',
137                     pattern: 'trivy-results.sarif'
138                 )
139             ]
140         )
141     }
142 }
143

```

```

144 // =====
145 // SONARQUBE ANALYSIS
146 // =====
147 stage('SonarQube Analysis') {
148     agent {
149         docker {
150             image 'sonarsource/sonar-scanner-cli:latest'
151             reuseNode true
152             args '--network cicd-network'
153         }
154     }
155
156     steps {
157         withSonarQubeEnv('SonarQube') {
158             sh '''
159                 set -e
160
161                 echo "=== SonarQube Analysis ==="
162
163                 sonar-scanner
164
165                 echo "=== SonarQube Analysis Completed ==="
166             '''
167         }
168     }
169 }
170
171 // =====
172 // QUALITY GATE
173 // =====
174 stage('Quality Gate') {
175     steps {
176         timeout(time: 30, unit: 'MINUTES') {
177             waitForQualityGate abortPipeline: true
178         }
179     }
180 }
181
182 // =====
183 // PACKAGE APPLICATION
184 // =====
185 stage('Package Application') {
186     agent {
187         docker {
188             image 'node:24-alpine'
189             reuseNode true
190             args '-u root'
191         }
192     }
193
194     steps {
195         sh '''
196             set -e
197
198             echo "=====
199             echo "          CREATING APPLICATION PACKAGE"
200             echo "=====
201
202             apk add --no-cache zip unzip

```

```

203
204     rm -f latest-app.zip
205
206     zip -r latest-app.zip . \
207         -x "node_modules/*" \
208         -x ".git/*" \
209         -x ".env" \
210         -x ".env.*" \
211         -x "coverage/*" \
212         -x ".trivy-cache/*" \
213         -x "latest-app.zip" \
214         -x "trivy-results.sarif"
215
216     echo "=== Application Package Created ==="
217
218     ls -lh latest-app.zip
219
220     echo "=== Package Content ==="
221
222     unzip -l latest-app.zip
223     ...
224 }
225 }
226
227 // =====
228 // PUBLISH APPLICATION
229 // =====
230 stage('Publish Application') {
231
232     steps {
233
234         // =====
235         // 1. ARCHIVE ARTIFACT KE JENKINS
236         // =====
237
238         archiveArtifacts(
239             artifacts: 'latest-app.zip',
240             fingerprint: true,
241             allowEmptyArchive: false
242         )
243
244         script {
245
246             // =====
247             // 2. BUAT IDENTITAS APPLICATION
248             // =====
249
250             def appName = env.JOB_NAME
251                 .replaceAll('[^a-zA-Z0-9._-]', '-')
252                 .replaceAll('-+', '-')
253                 .replaceAll('^|-$', '')
254
255             def buildId = env.BUILD_NUMBER
256
257             echo "Application Name: ${appName}"
258             echo "Build ID: ${buildId}"
259
260             // =====
261             // 3. COPY KE USER CONTENT

```

```

262 // =====
263
264 sh """
265     set -e
266
267     echo "=====
268     echo "          PUBLISHING USER CONTENT"
269     echo "=====
270
271     docker exec cicd-jenkins \
272         mkdir -p \
273
274 "/var/jenkins_home/userContent/applications/${appName}/${buildId}"
275
276     docker cp \
277         latest-app.zip \
278         "cicd-
jenkins:/var/jenkins_home/userContent/applications/${appName}/${buildId}/latest-app.zip"
279
280     echo "=== Published File ==="
281
282     docker exec cicd-jenkins \
283         ls -lh \
284
285 "/var/jenkins_home/userContent/applications/${appName}/${buildId}/latest-app.zip"
286
287 """
288
289 // =====
290 // 4. BUAT PUBLIC ARTIFACT URL
291 // =====
292
293 def jenkinsBaseUrl = env.BUILD_URL
294     .substring(0, env.BUILD_URL.indexOf('/job/'))
295     .replace('localhost', 'host.docker.internal')
296
297 env.ARTIFACT_URL =
298
299 "${jenkinsBaseUrl}/userContent/applications/${appName}/${buildId}/latest-app.zip"
300
301 echo "=====
302 echo "          APPLICATION PUBLISHED"
303 echo "=====
304
305 echo "Artifact URL:"
306 echo "${env.ARTIFACT_URL}"
307
308 }
309
310 }
311
312 // =====
313 // DEPLOY APPLICATION
314 // =====
315
316 stage('Deploy Application') {
317     agent {
318         docker {
319             image 'curlimages/curl:8.15.0'
320             reuseNode true
321             args '--network cicd-network'
322         }
323     }
324 }

```

```

317     }
318
319     steps {
320         script {
321
322             echo "=====
323             echo "          START APPLICATION DEPLOYMENT"
324             echo "=====
325
326             echo "Artifact URL:"
327             echo "${env.ARTIFACT_URL}"
328
329             // =====
330             // 1. REQUEST REDEPLOYMENT
331             // =====
332
333             echo ""
334             echo "=== Request Redeployment ==="
335
336             def redeployResponse = sh(
337                 script: '''
338                     set -e
339
340                     curl -sS --fail-with-body \
341                         -X POST "$URL_REDEPLOY" \
342                         -H "Content-Type: application/json" \
343                         -d "{
344                             \"token_access\\\": \"$DEPLOY_TOKEN\\\",
345                             \"website_id\\\": \"$WEBSITE_ID\\\",
346                             \"source_url\\\": \"$ARTIFACT_URL\\\",
347                             \"source_type\\\": \"jenkins\"
348                         }"
349                 ''',
350                 returnStdout: true
351             ).trim()
352
353             echo "Redeploy Response:"
354             echo redeployResponse
355
356             // =====
357             // 2. POLLING DEPLOYMENT PROGRESS
358             // =====
359
360             echo ""
361             echo "=== Waiting For Deployment ==="
362
363             def maxAttempts = 120
364             def attempt = 0
365             def deploymentStatus = 'IN_PROGRESS'
366
367             while (deploymentStatus == 'IN_PROGRESS') {
368
369                 attempt++
370
371                 if (attempt > maxAttempts) {
372                     error(
373                         "Deployment timeout. " +
374                         "Status masih IN_PROGRESS setelah " +
375                         "${maxAttempts} attempts."

```

```

376         )
377     }
378
379     sleep time: 5, unit: 'SECONDS'
380
381     echo ""
382     echo "=== Checking Deployment Progress (${attempt}/${maxAttempts})
===
383
384     def progressResponse = sh(
385         script: '''
386             set -e
387
388             curl -sS --fail-with-body \
389                 -X POST "$URL_PROGRESS" \
390                 -H "Content-Type: application/json" \
391                 -d "{
392                     \"token_access\": \"$DEPLOY_TOKEN\",
393                     \"website_id\": \"$WEBSITE_ID\"
394                 }"
395             '''
396         , returnStdout: true
397     ).trim()
398
399     echo "Progress Response:"
400     echo progressResponse
401
402     // =====
403     // PARSE JSON
404     // =====
405
406     def json = readJSON text: progressResponse
407
408     deploymentStatus = json?.data?.status
409         ?.toString()
410         ?.toUpperCase()
411
412     if (!deploymentStatus) {
413         error(
414             "Response progress tidak memiliki data.status"
415         )
416     }
417
418     echo "Deployment Status: ${deploymentStatus}"
419
420     // =====
421     // SUCCESS
422     // =====
423
424     if (deploymentStatus == 'SUCCESS') {
425
426         echo ""
427         echo "=====
428         echo "       DEPLOYMENT SUCCESS
429         echo "=====
430
431         break
432     }
433

```

```

434 // =====
435 // FAIL
436 // =====
437
438 if (deploymentStatus == 'FAIL') {
439
440     echo ""
441     echo "=====
442     echo "      ✖ DEPLOYMENT FAILED"
443     echo "=====
444
445     def deploymentLog =
446         json?.data?.log
447         ?: 'Deployment failed tanpa log.'
448
449     echo ""
450     echo "===== DEPLOYMENT LOG =====
451     echo deploymentLog
452     echo "=====
453
454     error(
455         "Deployment gagal untuk website " +
456         "${WEBSITE_ID}"
457     )
458 }
459
460 // =====
461 // OTHER STATUS
462 // =====
463
464 echo "Deployment masih berjalan..."
465 }
466
467 echo ""
468 echo "=====
469 echo "      ✔ DEPLOYMENT FINISHED"
470 echo "=====
471 }
472 }
473 }
474 }
475
476 // =====
477 // POST ACTIONS
478 // =====
479 post {
480
481     always {
482         archiveArtifacts(
483             artifacts: 'trivy-results.sarif',
484             allowEmptyArchive: true
485         )
486     }
487
488     success {
489         echo "=====
490         echo "      ✔ PIPELINE SUCCESS"
491         echo "=====
492         echo "Result: ${currentBuild.currentResult}"

```

```

493     echo "📦 Artifact: ${env.ARTIFACT_URL ?: '(not published)'}"
494     echo "🚀 Website berhasil dideploy."
495 }
496
497 failure {
498     echo "=====
499     echo "      ❌ PIPELINE FAILED"
500     echo "=====
501     echo "Result: ${currentBuild.currentResult}"
502     echo "Periksa log stage yang merah / Console Output untuk penyebab gagal."
503 }
504
505 unstable {
506     echo "=====
507     echo "      ⚠️ PIPELINE UNSTABLE"
508     echo "=====
509     echo "Result: ${currentBuild.currentResult}"
510 }
511 }
512 }

```

3. Modifikasi isi file `sonar-project.properties`, seperti berikut:

💖 Note

Ubah username `abdullah_ubaid` dengan username kamu. Contoh

`ifs18005`

🐾 Coding Time

```

1 sonar.projectKey=abdullah_ubaid-pabwe2026-reactjs
2 sonar.projectName=abdullah_ubaid-pabwe2026-reactjs
3
4 sonar.sources=src
5 sonar.tests=src
6 sonar.test.inclusions=**/*.test.js,**/*.test.jsx
7
8 sonar.sourceEncoding=UTF-8
9
10 sonar.javascript.lcov.reportPaths=coverage/lcov.info
11
12 sonar.exclusions=node_modules/**,coverage/**,dist/**,src/main.jsx,src/setupTests.js,src/test-
    utils.jsx,**/*.test.js,**/*.test.jsx

```

4. Push perubahan proyek ke Github. Pastikan file `Jenkinsfile` dan `sonar-`

project.properties terlihat pada repository Github.

The screenshot shows the GitHub repository interface for 'abdullah_ubaid-pabwe2026-reactjs'. The file list includes:

File	Commit Message	Time
public	Initial commit	4 hours ago
src	Initial commit	4 hours ago
.env	Initial commit	4 hours ago
.env.example	Initial commit	4 hours ago
.gitattributes	Initial commit	4 hours ago
.gitignore	Update .gitignore	4 hours ago
.oxlintrc.json	Initial commit	4 hours ago
Jenkinsfile	fetch: menambahkan konfigurasi CI/CD	now
README.md	Update README.md	4 hours ago
bun.lock	Initial commit	4 hours ago
index.html	Initial commit	4 hours ago
package.json	Initial commit	4 hours ago
sonar-project.properties	fetch: menambahkan konfigurasi CI/CD	now
vite.config.js	Initial commit	4 hours ago

1.5.2 Mengecek Jenkins

1. Buka Jenkins dan akses item sesuai repo Github yang di update. Karena sudah menggunakan webhook perubahan kode program akan otomatis dikirimkan ke Jenkins untuk dilakukan Job baru. Jika berhasil akan terlihat job baru, pilih pada job tersebut.

The screenshot shows the Jenkins job page for 'abdullah_ubaid-pabwe2026-reactjs'. The job status is 'Passed'. The 'Builds' section shows a list of builds, with the first build highlighted by a red arrow.

Build	Status	Time
#1	Passed	1:30 PM

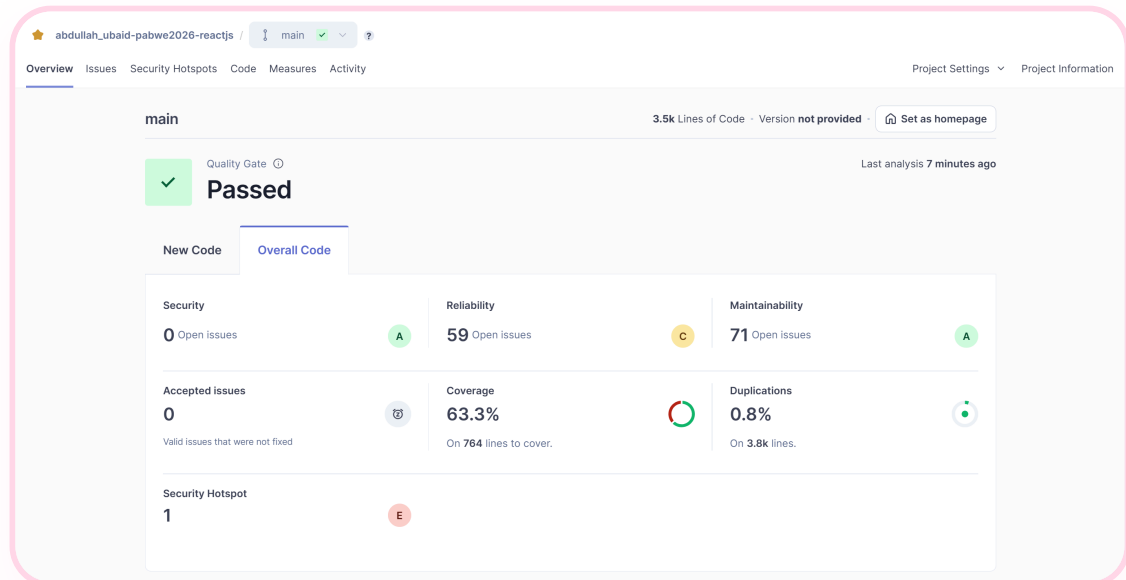
2. Pada halaman detail job Jenkins pilih menu **Pipeline Overview**.

3. Tampilan jika job berhasil di-build.

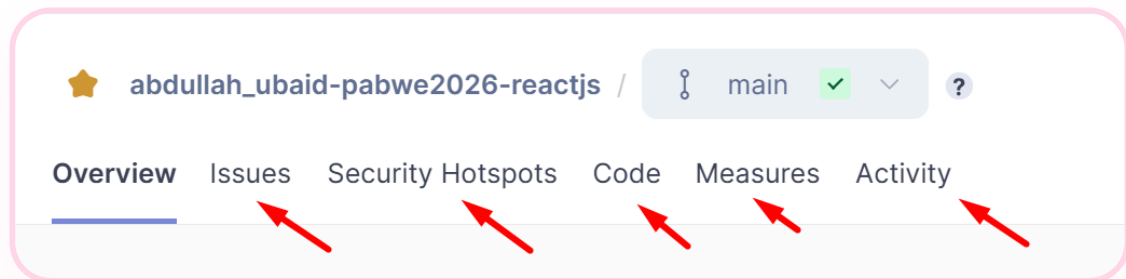
1.5.3 Mengecek SonarQube

1. Buka SonarQube dan akses proyek sesuai item jenkins yang di-build. Karena sudah menggunakan webhook perubahan akan otomatis dikirimkan ke SonarQube untuk dilakukan analisis kode program. Berikut tampilan overview analisis kode program di

proyek SonarQube:

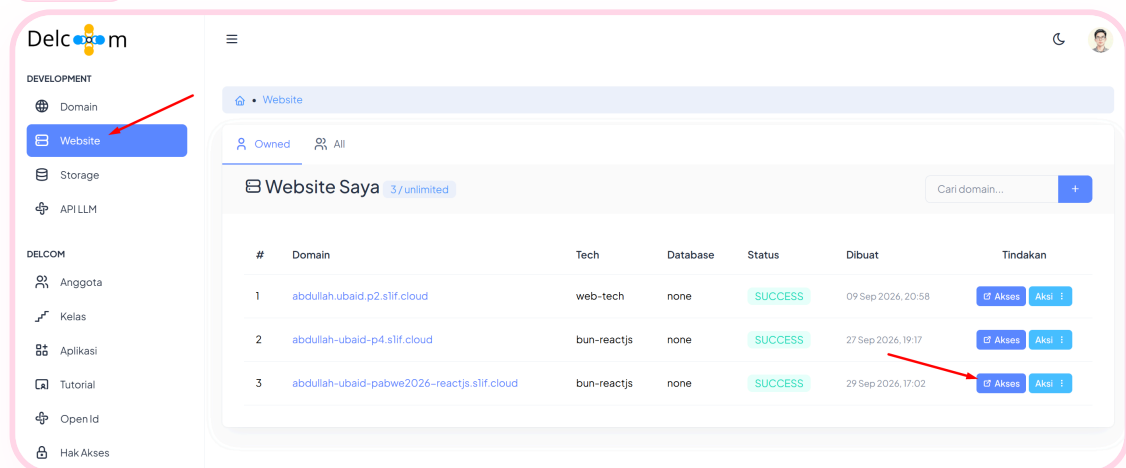


2. Kamu bisa membuka tab satu persatu untuk melihat detail hasil analisis kode program.



1.5.4 Membuka Aplikasi Web

1. Buka delcom dan akses menu **website**. Pada halaman website pilih tombol **Akses** pada website yang ingin dibuka.



2. Jika berhasil tampilan web akan terlihat.

Sebelum login



Delcom Lost & Found

Platform Pencatatan Barang Hilang dan Temuan

Masuk Akun
Daftar Baru

ALAMAT EMAIL

 nama@email.com

KATA SANDI



 **Masuk Sekarang**

Sesudah login


Delcom Lost & Found


Abdullah Ubaid
aud@delcom.org

MENU UTAMA

-  Lost & Found
-  Semua Pengguna
-  Profil Saya

Pusat Lost & Found

Lacak barang hilang atau laporkan penemuan barang di lingkungan kampus.

TOTAL LAPORAN

1

BARANG HILANG

1

BARANG DITEMUKAN

0

TELAH SELESAI

0

Semua Hilang Temuan
Semua Status Proses Selesai
Laporan Saya

ID	BARANG	JENIS	PELAPOR	TANGGAL	STATUS	AKSI
#1	Baju Baju almamater ukuran L	Kehilangan	Abdullah Ubaid	29 September 2026 pukul 13.34	Proses	  

B. Submission CI/CD

2.1 Jenkins Grading

- Buka konten kursus Jenkins Grading.

Menu Kursus
Konten
Submission


Aplikasi Lost&Founds - ReactJS


Jenkins Grading
06 October 2026 - 23:59

Status Pengumpulan

Kamu belum mengumpulkan jawaban!

Jumlah Percobaan: 0 / Tidak Terbatas

Waktu tersisa: 07 hari 02j 55m 25d

 Sinkronisasi Hasil

2. Scroll ke bawah perhatikan kriteria submission terkhususnya untuk nama job jenkins dan repo github.

Informasi	
Score Akhir	100 /100
Nama Job (setelah render)	abdullah_ubaid-pabwe2026-reactjs
Nama Repo Git	abdullah_ubaid-pabwe2026-reactjs
Jenkinsfile (template)	Ada (dicek SHA256)
sonar-project.properties	Ada (dicek SHA256)

3. Scroll ke bawah, jika konten file **Jenkinsfile** atau **sonar-project.properties** tersedia pastikan menggunakan konten file tersebut pada tahapan **1.5.1**

Jenkinsfile & sonar-project.properties
Sudah di-render untuk akun abdullah_ubaid {{{username}}} diganti)

Jenkinsfile **rendered**

```

1 pipeline {
2
3   agent any
4
5   options {
6     timestamps()
7     skipDefaultCheckout(true)
8     // Stop later stages if a publisher marks the build UNSTABLE/FAILURE mid-run
9     skipStagesAfterUnstable()
10  }
11
12  stages {
13
14    // =====
15    // CHECKOUT
  
```

sonar-project.properties **rendered**

```

1 sonar.projectKey=abdullah_ubaid-pabwe2026-reactjs
2 sonar.projectName=abdullah_ubaid-pabwe2026-reactjs
3
4 sonar.sources=src
5 sonar.tests=src
6 sonar.test.inclusions=/**/*.test.js,/**/*.test.jsx
7
8 sonar.sourceEncoding=UTF-8
9
10 sonar.javascript.lcov.reportPaths=coverage/lcov.info
11
12
  
```

4. Jika pada tahapan **1.5.2** sudah berhasil, pilih tombol **Sinkronisasi Hasil** pada konten kursus Jenkins Grading.

Menu Kursus

Konten Submission

Aplikasi Lost&Founds - ReactJS

Jenkins Grading 06 October 2026 - 23:59

Status Pengumpulan

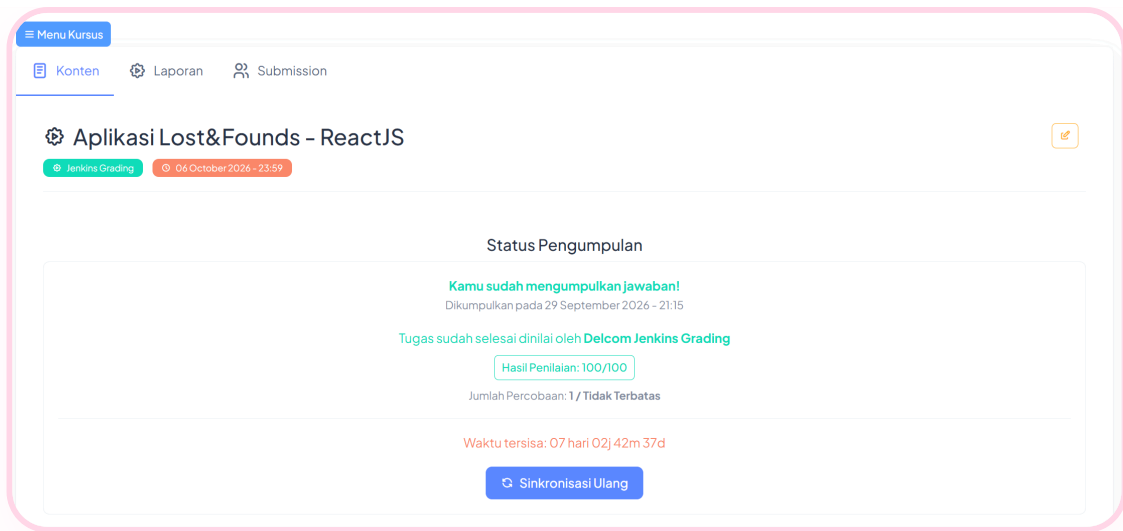
Kamu belum mengumpulkan jawaban!

Jumlah Percobaan: 0 / Tidak Terbatas

Waktu tersisa: 07 hari 02j 53m 16d

Sinkronisasi Hasil

5. Jika jenkins sesuai dengan ketentuan akan terlihat nilai yang diperoleh.

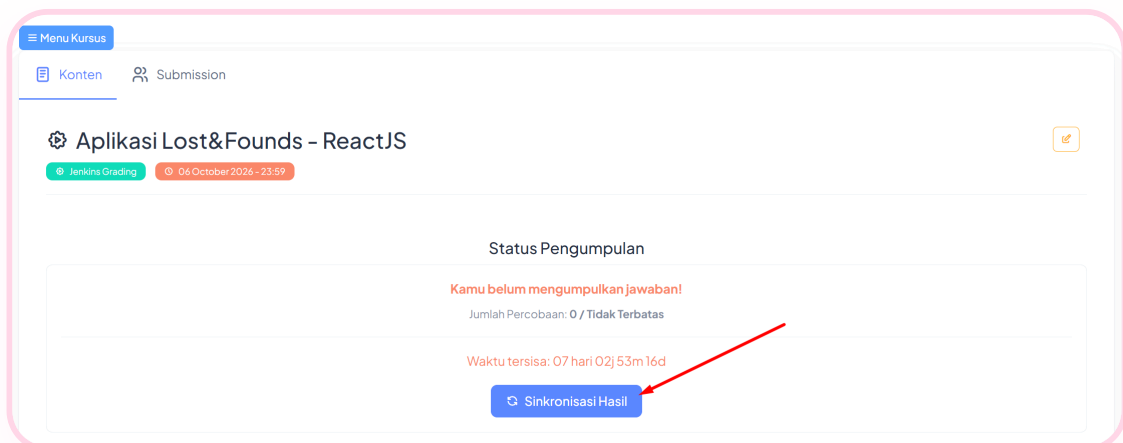


2.2 SonarQube Grading

1. Buka konten kursus **SonarQube Grading**.
2. Scroll ke bawah perhatikan kriteria submission terkhususnya untuk nama proyek.



3. Jika pada tahapan **1.5.3** sudah berhasil, pilih tombol **Sinkronisasi Hasil** pada konten kursus SonarQube Grading.



4. Jika SonarQube sesuai dengan ketentuan akan terlihat nilai yang diperoleh.



🐾 🌸 ✨ Happy Coding • Happy Learning • Happy Life ✨ 🌸 🐾